Assignment 1: Language ID

- Due Thursday by 11:59pm
- Points 100
- Submitting an external tool
- Available after Sep 3 at 12:01am

Hi Everyone!

We have created a Gradescope assignment for the same. Please submit your completed Ipython Notebook (HW1.ipynb) in that Gradescope assignment.

Introduction

In this assignment, you will build a language identification classifier that distinguishes between six languages:

- **Hausa (https://en.wikipedia.org/wiki/Hausa_language)**
- **Indonesian (https://en.wikipedia.org/wiki/Indonesian_language)**
- **Manobo (https://en.wikipedia.org/wiki/Manobo_languages)**
- **Nahuatl (https://en.wikipedia.org/wiki/Nahuatl)**
- **Swahili (https://en.wikipedia.org/wiki/Swahili_language)**
- **Tagalog (https://en.wikipedia.org/wiki/Tagalog_language)**

Some languages can be distinguished easily, because they use different scripts. These six languages, however, use the same (**Latin (https://en.wikipedia.org/wiki/Latin_script)**) script with minimal **diacritics (https://en.wikipedia.org/wiki/Diacritic)** so it is difficult to hand-craft classifiers based on the presence or absence of particular characters. Indeed, unless you have linguistic training or familiarity with the languages, it is difficult to tell them apart.

How can they be distinguished? A naïve approach is to use word counts as unigram features. However, the number of possible words in a large corpus of five languages is vast. It is essential to look at something smaller — characters.

Even though the six languages use roughly the same characters, the relative frequencies of these characters vary greatly. Thus, using characters as features (unigram character models) is appealing (and fairly effective). It is also true that languages very in their *phonotactics*, the way in which consonants and vowels combine in sequence. Thus, looking at character ngrams (for small

values of *n*) is also appealing (and effective). Note, however, that as the value of *n* increases, this approach runs into the same problem as the word unigram model (sparcity). In this scenario, the model is likely to overfit.

Various kinds of classifiers can be used for this application. NB classifiers, for example, are quite effective. However, inference is slow and performance, given the same training set, is likely to be worse than other options. Simple logistic regression cannot be used because this is an n-way (multinomial) classification problem. Multinomial Logistic Regression (Softmax Regression) is a good fit.

Summary

You will perform the following tasks:

1. Implement a training loop for Multinomial Logistic Regression.
2. Implement inference for Multinomal Logistic Regression
3. Determine the optimal order of *n* for ngrams for MNLR trained on the training set.
4. Calculate and display a confusion matrix for a trigram model evaluated on the test set.
5. Inspect the feature weights, and display the most predictive features for each language.

HW1.ipynb (https://canvas.cmu.edu/courses/53604/files/14736739?wrap=1)
(https://canvas.cmu.edu/courses/53604/files/14736739/download?download_frd=1)
train.tsv (https://canvas.cmu.edu/courses/53604/files/14736742?wrap=1)
(https://canvas.cmu.edu/courses/53604/files/14736742/download?download_frd=1)
test.tsv (https://canvas.cmu.edu/courses/53604/files/14736741?wrap=1)
(https://canvas.cmu.edu/courses/53604/files/14736741/download?download_frd=1)
confusion_matrix.png (https://canvas.cmu.edu/courses/53604/files/14736743)

This tool needs to be loaded in a new browser window

Load Assignment 1: Language ID in a new window